McMaster University, DATASCI 3ML3 January 9 - January 23 2026
http://avenue.mcmaster.ca/

Homework 1 :
Optimization, Coordinate descent

Due: Friday January 23 2025, 11:30pm on CROWDMARK, with a grace period till the following Monday at 11:30pm

☛**Notes:**

- If you find that your current computer is not up to running jupyter notebooks with python, we have a working jupyter hub we can give you access to. You can then access jupyter through any web-browser. Contact your TA if you find that this will be needed. The jupyter hub is accessible at `http://physics.mcmaster.ca/jupyter`. Don't use `https://mcmaster.syzygy.ca/` as it doesn't have the required libraries installed and you won't have a home directory persisting between sessions.
- You are encouraged to discuss the exercises with your classmates but the work you hand in must be your own.
- Upload your solutions to crowdmark in the format specified there.
- To start we will mainly be using python 3.8.12 and matplotlib 3.4.3 but later on we will need to use other versions of python. Read about setting up virtual environments here:
  `https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments`
  or here:
  `https://docs.python.org/3/library/venv.html`
  or here:
  `https://realpython.com/python-virtual-environments-a-primer/`.
- Files for this homework are available on avenue.

☛**Reading:** This week and next we will focus on mathematical optimization following Watt, Borhani and Katsaggelos *Machine Learning Refined* (MLR). The textbook is available in the bookstore and drafts are available on github `https://github.com/jermwatt/machine_learning_refined`. Many of the exercises in HW1 are modifications of some of the problems in Chapter 2 of MLR.

Exercise 1 *Using the MLR libraries for plotting*

In all the exercises you may use the libraries provided by the book (MLR) for plotting. It is much easier for marking and debugging if we all use the same thing. However, the libraries are a bit quirky and sometimes don't work so you're more than welcome to roll your own plotting routines. To get started with using the plotting library:

1. Download the zip file MLRPlotting.zip from avenue.
2. unzip it in the directory where you want to work
3. Test that you can use the library. You may have to provide the path to where you put the library by uncommenting the first 2 lines in HW1.Plotting.py, and filling in the path on your computer.

To demonstrate that the library is working for you, submit the plot of the output of the following script (HW1.Plotting.py on avenue). You of course have to fill in your actual First Name and Last Name.

```
#import sys
#sys.path.append('Path to where you put MLRPlotting')
import matplotlib.pyplot as plt
from MLRPlotting import static_plotter

# import custom plotting tools
static_plotter = static_plotter.Visualizer();

cost_1 = [1,2,7,9]
cost_2 = [9,6,3,1]
# plot the cost function history for two runs
static_plotter.plot_cost_histories([cost_1,cost_2],
        start=0,points=True,labels=['First Name','Last Name'])
```

Exercise 2 *Random Numbers and numpy*

In preparation for the subsequent exercise, we'll play around with random numbers and numpy a bit in this exercise. We begin with random numbers. First we `import numpy as np`.

2.1 Use `np.random.randn` to generate a set of `num_samples` random vectors of length `D`. Let's call the results `directions` and you should make sure it has `num_samples` rows and `D` columns. What kind of random numbers are the elements of `directions` ? Does each row correspond to a unit vector ?

2.2 The way you have generated `directions` means that it is a numpy object which implies additional functionality. What does the operation `directions*directions` result in ? We want to explicticly normalize each row in `directions` and in order to do that we need to be able to partially sum the result of `directions*directions`. What is the result of the operations

```
np.sum(directions*directions)
np.sum(directions*directions,axis=0)
np.sum(directions*directions,axis=1)}
```

Which one do we need if we want to normalize the rows ?

2.3 Let's call the answer to the last point in the preceeding question `psum`. It should contain the sum of the square of the elements of each of the random vectors. To normalize them we simply need to do `np.sqrt(psum)`. Now, it would be great if could simply normalize the rows of `directions` using the statement:

```
directions = directions/norms
```

Try it. It doesn't work because of what in numpy is called broadcast rules, which we will discuss in more details later. In essence, numpy doesn't know what to do because it doesn't know enoguh about the shape of `norms`. We can fix that several different ways. For instance, we can do `norms.shape=(num_samples,1)` or `norms=norms[:,newaxis]`. Using this, verify that each row of `directions` is normalized.

Exercise 3 *Random Search and Rosenbrock Banana function*

In this exercise we you will use an implementation of the random search algorithm that MLR provide on their github pages. You can find it on avenue. The beginning look like this:

```
# random search function
def random_search(g,alpha_choice,max_its,w,num_samples):
    # run random search
    w_history = []         # container for w history
    cost_history = []      # container for corresponding cost function history
    alpha = 0
    for k in range(1,max_its+1):
        # check if diminishing steplength rule used
        if alpha_choice == 'diminishing':
            alpha = 1/float(k)
        else:
            alpha = alpha_choice
            .
            .
            .

    weight_history.append(w)
    cost_history.append(g(w))
    return w_history,cost_history
```

Here, `g` is the function we're trying to minimize. `alpha_choice` is the step length/learning rate which can take 2 values, either the string `'diminishing'` or simply a numerical constant. `max_its` is the maximum number of iterations to be taken and `w` the starting vector. `num_samples` is the number of random directions that we choose for each iteration. In addition we like to be able to monitor the progression of the algorithm so you should keep track of the points the algorithm visits in `w_history` as well as the associated cost in `cost_history`, where by 'cost' we simply mean what value the function has in the point we're currently at. The implementation provided follows closely what you did in the preceeding exercise. Take some time to read through the code and understand all the steps. The book has a particular python style that many of the examples follow so if you understand this one you'll be able to read many of the other examples as well.

Note that the `directions` are normalized and that `alpha` is step length. If `directions` contain *all* the generated and normalized directions we can then generate *all* the candidates (`w_candidates`) for new points simply by using `w_candidates = w + alpha*directions` as implemented in the code.

3.1 Now we want to test the `random search` function. We want to find the minimum of the function wich is simply:

$$g(\vec{w}) = \mathbf{w}^T\mathbf{w} + 2 \;\; (= \vec{w} \cdot \vec{w} + 2) \tag{1}$$

in python we can implement this in a straight forward manner as shown below. In addition, we start the search with the following parameters:

```
g = lambda w: np.dot(w.T,w) + 2
alpha_choice = 1; w = np.array([3,4]); num_samples = 1000; max_its = 5;
```

Here, `w = np.array([3,4])` specify the starting point for the minimization. Show plots of the resulting `w_history,cost_history`. We have already seen how you can plot `cost_history` in the first exercise. There is also a simple way of plotting `w_history`. You can do that using
`static_plotter.two_input_surface_contour_plot(g,w_history,view = [10,30],xmin = .., xmax =.., ymin = .., ymax = ..,num_contours = 20)`
where you have to fill in reasonable values for `xmin` etc. Have a look at what you did in exercise

1, you again have to import the class Visualizer.

3.2 Repeat the analysis but with a steplength of `alpha=0.3`. Did the results improve ? Explain your results.

3.3 Next we want to study a function in two dimensions with a very long, narrow and curved valley, the Rosenbrock Banana function:

$$g(w_1, w_2) = 100(w_2 - w_1^2)^2 + (w_1 - 1)^2 \tag{2}$$

This function has a minimun at $(w_1, w_2) = (1, 1)$. Make two runs with `num_samples=1000;max_its = 50;` starting from the point $(-2, -2)$. First run with a fixed step length of $\alpha = 1$ then with a diminishing steplength $\alpha = 1/k$. Compare the two runs by plotting the `cost_history`. Also make contour plots of the $g(w_1, w_2)$ with an overlayed `w_history`. If you want a nice plot you can again try using `static_plotter.two_input_surface_contour_plot` from MLRplotting.

3.4 Now let's try something a little more challenging, the six-hump camel function:

$$g(w_1, w_2) = (4 - 2.1w_1^2 + w_1^4/3)w_1^2 + w_1w_2 + (-4 + 4w_2^2)w_2^2. \tag{3}$$

In the domain $w_1 \in [-3, 3]$, $w_2 \in [-2, 2]$ it has 2 global minima at $(0.0898, -0.7126)$ and $(-0.0898, 0.7126)$. Select a starting point and use random search to locate the minima. Do you converge to a global minima or stuck elsewhere ?

Exercise 4 *Curse of Dimensionality*

Let us explore the curse of dimensionality using the above $g(\mathbf{w}) = \mathbf{w}^T\mathbf{w} + 2$. We start with an initial vector $\mathbf{w}^0 = (1, 0, \ldots, 0, 0)$ but the length of the vector (the dimension of the space) is no longer 2. Instead let's call it N. We're not going to search for the minimum but instead we shall just just look around the neighborhood of $\mathbf{w}^0$ and determine how many of P randomly generated directions are descent directions. For $N = 1, 2, \ldots, 25$ make a plot of the *fraction* of the P directions that are descent directions. Make a plot of the fraction versus N for the 4 values of $P = 10, 100, 1000, 10000$. Comment on your observations.

Exercise 5 *Coordinate Search and Coordinate Descent*

In this exercise we will look at coordinate search and coordinate descent. Please read section 2.6 in MLR. In high dimensions random search is no longer efficient. We therefore want to look at coordinate search and coordinate descent. For coordinate search, instead of generating P (or `num_samples`) random directions we only look for descent among the $2N$ coordinate directions. That means that the candidate directions to go to are always the same and can be fixed at the start (have a look at `np.eye`). For instance like this:

```
# construct set of all coordinate directions
directions_plus = np.eye(np.size(w),np.size(w))
directions_minus = - np.eye(np.size(w),np.size(w))
directions = np.concatenate((directions_plus,directions_minus),axis=0)
```

For coordinate search we go through all of these directions at every step and select the best one.

For coordinate descent we instead *randomly* go through the coordinate directions and the first time we find a descent we use it.

☛**Note:** The way that coordinate search works is an example of a *greedy* strategy. You always pick the best. Try reflecting on why that might not always be the best strategy, and when it might actually be a good strategy.

5.1 Modify your implementation of `random_search` to instead implement `coordinate_search` and `coordinate_descent`. In both cases you should use normalized directions (multiplied by the steplength). Make sure you can go both in the postive and negative direction for each coordinate. In one dimension there would be two directions, forward and backward. In two dimensions there would be 4 directions, north, south, east and west.

☛**Hint:** For coordinate descent have a look at `numpy.random.permutation`

☛**Watch out:** For a fair comparison between the 2 algorithms you should allow `coordinate_descent` to *attempt* to take N steps whenever `coordinate_search` takes one step. One way to do that is to generate a random permutation of the $2N$ directions and let `coordinate_descent` run through them sequentially before generating another permutation, letting `max_its` count the number of permutations generated. So, if `max_its` is the maximum number of steps that `coordinate_search` can take then `coordinate_descent` could *possibly* take $N \times$`max_its` step. Read section 2.6.2 in the text book.

If you get stuck, look at the implementation MLR provide on their github page, but try to do it yourself first.

5.2 Consider again the Rosenbrock Banana function:

$$g(w_1, w_2) = 100(w_2 - w_1^2)^2 + (w_1 - 1)^2 \tag{4}$$

Starting from $\mathbf{w}^0 = (-2, -2)$, using a maximum number of itereations $P = 40$ and a diminishing alpha, compare coordinate search with coordiate descent by plotting the cost history.

Exercise 6 *Gradient Descent*

Complete the notebook on autograd and Gradient Descent labelled HW1.6.ipynb on avenue.

☛**Note:** It is often useful to know which version of python and matplotlib you're using. The following commands will tell you:

```
from platform import python_version
print(python_version())
import matplotlib
print(matplotlib.__version__)
```